

SGP: Sistema Gestione Personale

Studente: *Matteo Giorgio* – Classe: *5ª B Informatica* – Anno Scolastico: *2025/2026*

Repo github: <https://github.com/zMqTTew/Sistema-di-Gestione-Personale-SGP->

Tutorial YT: <https://youtu.be/YugSHwJmpIY>

1. Introduzione e analisi del problema

L'idea alla base di questo progetto nasce da un'esigenza concreta e quotidiana. Utilizzando intensamente il computer per lo studio e lo sviluppo, mi sono reso conto di quanto tempo utile perdessi a causa del disordine: file scaricati e accumulati sul desktop, documenti sparsi e cartelle caotiche rendevano spesso le sessioni di lavoro frustranti. Inoltre a questo si aggiungeva l'idea di gestire in modo sicuro e centralizzato le mie credenziali d'accesso ai vari servizi web, senza dipendere da soluzioni esterne di cui non ho il pieno controllo.

Invece di affidarmi a software commerciali di terze parti, ho colto questa necessità come un'opportunità di crescita. Ho deciso di sfruttare le competenze acquisite durante il mio percorso scolastico per realizzare un software mio, personale, che potessi utilizzare concretamente ogni giorno. Per farlo ho voluto fare un passo oltre il programma scolastico, documentandomi e integrando nel progetto librerie specifiche che non avevamo mai applicato in classe (come *CustomTkinter* per l'interfaccia grafica o *Cryptography* per la sicurezza).

2. Obiettivi del software

L'obiettivo principale di **SGP (Sistema di Gestione Personale)** è centralizzare in un'unica applicazione desktop leggera, autonoma e portabile (compilata in un unico file `.exe`) tre funzionalità chiave per l'ottimizzazione dell'esperienza utente:

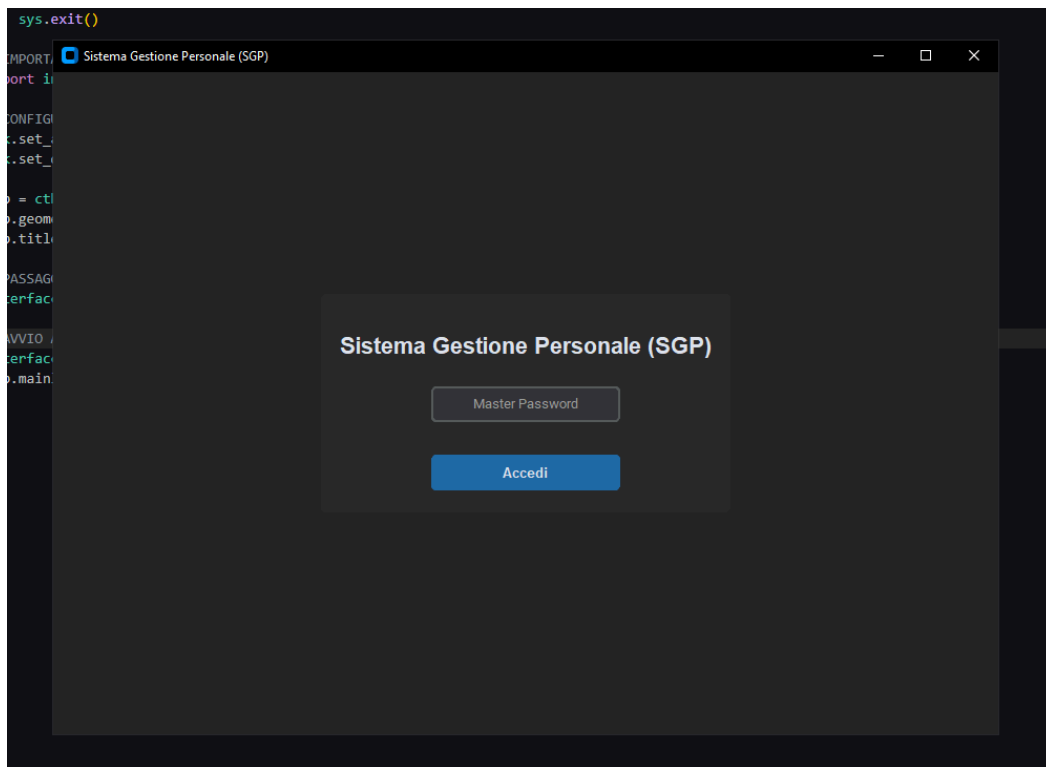
1. **Automazione del File System:** scansione e riordino istantaneo di qualsiasi directory basato sulla categorizzazione rigida delle estensioni dei file.
2. **Sicurezza Locale Crittografata:** creazione di un semplice "registro" digitale per la memorizzazione protetta delle password, basato su standard crittografici e operante offline per garantire una buona privacy.
3. **Monitoraggio delle Risorse:** analisi visiva immediata delle prestazioni dell'hardware principale (CPU, RAM e Disco) per tenere sotto controllo la salute del sistema durante l'uso.

3. Architettura del software e scelte tecniche

L'applicazione è stata suddivisa in quattro moduli distinti e interconnessi:

- `main.py`: il punto di ingresso del programma, verifica la presenza dei requisiti hardware/software, inizializza la finestra principale e avvia il loop dell'applicazione.
- `interfaccia.py`: gestisce interamente la GUI, contiene la logica visiva di tutte le finestre, dei pulsanti d'azione e lo switch dinamico tra i vari menu.
- `sicurezza.py`: il nucleo crittografico. Gestisce le operazioni di hashing, la cifratura del "database" e la scrittura dei log di sistema.
- `gestore_file.py`: contiene le regole per la scansione delle directory e il posizionamento organizzato dei file.

Per l'interfaccia grafica, ho scelto **CustomTkinter** rispetto al classico *Tkinter* studiato a scuola, poiché permette di generare widget moderni con supporto nativo alla Dark Mode e una resa estetica decisamente più professionale. Per il monitoraggio, la libreria **psutil** mi ha permesso di interfacciarmi direttamente con il sistema operativo per estrarre le metriche in tempo reale.



4. Scelte sulla sicurezza e gestione dati

La sicurezza del modulo Password Manager è l'aspetto su cui ho applicato la teoria di sicurezza informatica studiata in Sistemi e Reti.

Durante il percorso scolastico abbiamo lavorato spesso con database relazionali attraverso ambienti locali come XAMPP, utilizzando PHP e MySQL. Tuttavia per questo progetto specifico, la mia idea era creare un'applicazione totalmente indipendente, portabile e leggera, che potesse girare su qualsiasi computer senza richiedere l'installazione o l'attivazione di server locali in background. Per questo motivo, ho deciso di documentarmi sul funzionamento dei file JSON (`vault.json`), preferendo questo formato specifico per la sua applicazione basica e immediata, infatti la struttura a dizionario chiavi-valore del JSON si è rivelata perfetta per gestire la persistenza dei dati a runtime in modo leggero.

Il livello di sicurezza è garantito da tre barriere:

1. **Protezione della Master Password:** la password di accesso alla suite non viene mai salvata in chiaro. Al momento del login, il sistema genera un hash *SHA-256* della stringa inserita e lo confronta con quello memorizzato.
2. **Derivazione della chiave:** per rendere sicura la crittografia, ho utilizzato il protocollo *PBKDF2HMAC* che, partendo dalla password e da un *salt* casuale generato dal sistema operativo (`os.urandom`), deriva una chiave crittografica forte a 32 byte.
3. **Cifratura AES-256:** le credenziali dei singoli siti vengono salvate nel JSON cifrate tramite l'algoritmo simmetrico *AES a 256 bit* (implementazione *Fernet*), in questo modo anche se una persona esterna aprisse il file `vault.json`, leggerebbe solamente stringhe indecifrabili.

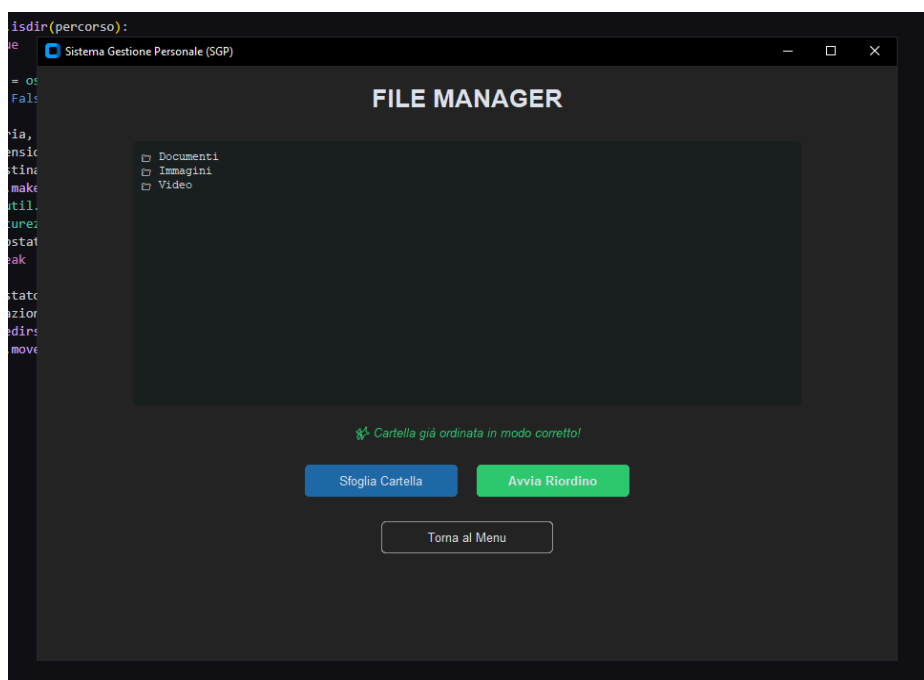
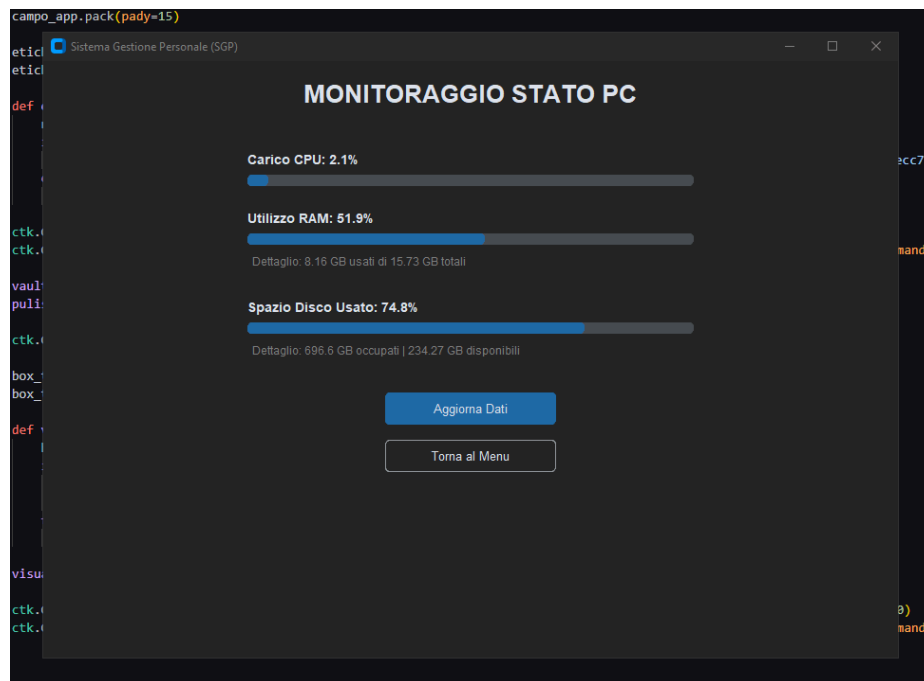
5. Alcune fasi dello sviluppo

Il percorso di sviluppo non è stato lineare e mi ha costretto a mettermi in gioco più volte:

- **Fase 1: Logica e Crittografia.** Ho iniziato scrivendo le funzioni crittografiche e di riordinamento dei file in modalità testuale (senza grafica), per assicurarmi che gli algoritmi di

crittografia e di spostamento file (`shutil.move`) funzionassero in modo corretto senza corrompere i dati.

- **Fase 2: L'idea iniziale della grafica e il cambiamento.** La mia idea iniziale era creare un'interfaccia standard con Tkinter, ma il risultato visivo era arcaico e poco moderno. Volevo qualcosa di moderno e navigando online, ho scoperto *CustomTkinter*, che mi ha permesso di realizzare un design basico ma comunque moderno.
- **Fase 3: I problemi con lo Stato PC.** Quando ho implementato il monitor delle risorse, l'interfaccia grafica si bloccava per un secondo ogni volta che aggiornavo i dati della CPU, perché il comando di psutil di default attendeva un intervallo di campionamento. Sfruttando l'AI come supporto al debug, sono riuscito a individuare la soluzione giusta: impostare il parametro `interval=None` e ottimizzare le istruzioni di processing dei dati, risolvendo i micro-freeze e rendendo i grafici a barra fluidi e reattivi.
- **Fase 4: Compilazione.** Infine ho utilizzato *PyInstaller* per impacchettare i moduli e le risorse grafiche in un unico file `.exe`, rendendo l'applicazione portabile su qualsiasi PC Windows senza installare Python.



6. Conclusioni e sviluppi futuri

Realizzare **SGP** è stata una sfida personale molto stimolante. Vedere concetti studiati sui libri di testo – come la crittografia simmetrica, l'hashing, l'I/O su file system e la gestione delle risorse hardware – prendere vita in un software reale, scritto da me e realmente funzionante sul mio computer, mi dà una soddisfazione enorme.

Il progetto è una base solida, ma non lo considero ancora chiuso. Per il futuro, ho intenzione di espandere la sicurezza del Password Manager inserendo un **generatore automatico di password robuste** basato sulla libreria `secrets`, evitando così che sia l'utente a dover inventare chiavi complesse.

Per quanto riguarda il modulo di monitoraggio hardware, un'evoluzione fondamentale sarà l'inserimento delle **statistiche in tempo reale della GPU**. Durante lo sviluppo iniziale ho riscontrato alcune limitazioni e incompatibilità con le librerie standard (come le dipendenze hardware specifiche richieste da alcune estensioni di `psutil` o librerie terze), che mi hanno spinto a rimandare la funzione per non compromettere la stabilità del software.